



DESIGN

Architecture & Design

How the system is built, and why each decision was made — written for an engineer or a judge who wants to understand the design, not just run it.

The problem, as engineering constraints

The brief has two halves — turn audio into text, then build intelligence — but the *scoring* is where the real constraints hide:

REQUIREMENT IN THE BRIEF	THE CONSTRAINT IT IMPOSES
"work out who said what"	Speaker attribution must be correct , not approximate — a wrong speaker poisons every downstream judgment.
"the mood ... and the point where it shifted"	Every judgment needs a timestamp , so the transcript must carry real timings.
"no evidence scores zero; wrong evidence scores negative"	Judgments must be grounded and verifiable — safer to check the evidence than to trust the model.
"do not re-transcribe on every request"	Analysis must be precomputed and stored ; the API is a read layer.
"run the whole thing ... from scratch"	The system must be reproducible with no secrets — degrade gracefully with no API key.

System overview



The five load-bearing decisions

1 • Channel-split diarization, not an ML diarizer

The recordings are stereo — agent on the left channel, customer on the right. Rather than run a speaker-diarization model to *guess* who spoke (the usual approach, and a

common source of error), we split the channels with ffmpeg and transcribe each independently, then interleave by timestamp. Speaker attribution becomes correct **by construction**. We substantiate it with a number (cross-channel correlation $\approx -0.13 \rightarrow$ 100% separation), and auto-correct the ~37 reversed-channel recordings.

2 • Evidence as a first-class, verified object

Every judgment carries `{timestamp, verbatim quote, speaker}`, and a verification pass checks each quote actually appears in the transcript at that timestamp. Unverified citations render in amber. This mirrors exactly where the 2025 summarization-faithfulness literature (FaithBench / FaithJudge) has moved: measure whether a claim is *faithful to the source*, not whether it matches a gold string.

3 • A pluggable analysis engine

Four interchangeable backends chosen at runtime: `claude`, `azure` (GPT-4o / GPT-5-mini), `azure-claude` (Sonnet 5), `ollama`, and a dependency-free `heuristic`. A judge with no API key runs the heuristic and still gets a complete, evidence-cited, 100%-coverage system. Structured output is guaranteed on both LLM paths via a forced tool call whose schema is the analysis schema.

4 • QA / compliance — "sounded resolved but wasn't"

On top of the analysis, `qa.py` scores each call against a bank-grade rubric from the transcript: identity verified before a sensitive action? action confirmed? empathy? repeated question? professional open/close? It flags **resolution-risk** calls that closed politely but were never actually completed — the exact failure the problem statement names.

5 • Precompute-and-store; the API is a thin read layer

The pipeline writes transcript + analysis + QA into a single PostgreSQL `calls` table. The FastAPI backend only reads — it never transcribes or calls an LLM on a request.

Design trade-offs

CHOICE	ALTERNATIVE	WHY WE CHOSE IT
Channel-split diarization	ML speaker diarizer	Correct by construction; cheaper; measurable.
Verify evidence post-hoc	Trust the model's citation	The scoring penalises wrong evidence; checking is cheap insurance.
Faithfulness + coverage metrics	A single "accuracy %"	Exact-match is brittle and punishes good-but-reworded answers.
Single wide <code>calls</code> table	Normalised schema	Matches how data arrives; simpler ingestion; fast enough at this scale.
Precompute to Postgres	Analyse on request	The brief forbids per-request transcription; better latency.
Pluggable engine w/ heuristic floor	LLM-only	Guarantees "runs from scratch" with no key.

Companion for CallRadar — Architecture & Design. See also Formulas & notation, the complete documentation.